面向 NAB 多场景业务指标的周期感知与互补融合异常检测

课程名称：数据仓库与数据挖掘

小组成员：第 9 组，李思懿（组长）、翁汇坤

1. 背景与问题定义

1.1 应用场景与数据构成

在云基础设施监控、城市运行感知与互联网舆情监测场景中，业务性能指标通常以单变量时间序列形式持续写入时序数据库。本项目面向 Apache IoTDB 2.0.11 树模型下的单变量流式异常检测任务，目标是在“先预测评分、后更新状态”的因果条件下，兼顾不同周期强度与异常持续形态，在控制误报的同时保持窗口检出能力。

项目早期原型仅在 NYC Taxi 单一序列按旧日期切分的 5 个异常窗口上检验基础周期方法。为扩展评估范围，本项目采用 Numenta Anomaly Benchmark（NAB）[11]，固定上游版本 ea702d75cc2258d9d7dd35ca8e5e2539d71f3140，使用 MIT 许可。从全部 47 条真实候选出发，按四项规则筛选：观测值均有限、时间戳唯一且构成完整规则采样网格、步长整除一天，以及前 40% 训练段至少覆盖 4 个日周期加 1 点。筛选不插补缺失值，不依据测试成绩。最终纳入 21 条、排除 26 条；其中 25 条网格不完整、7 条训练历史不足，排除原因存在重叠。

入选序列共 223,878 个实测点，按时间划分为 40% 训练、20% 校准与 40% 测试，测试段共 89,559 点，覆盖同一基准下的三类场景：

- AWS 云监控：9 条、8 个测试窗口。 包括 EC2 与 RDS 的 CPU 利用率、磁盘写入量及自动伸缩应用监控序列。
- 已知原因序列：2 条、5 个测试窗口。 包括纽约出租车需求和自动伸缩集群 CPU 指标。当前划分下 NYC 为 4 个测试窗口，与早期旧日期划分的 5 窗不混计。
- Twitter 推文计数：10 条、9 个测试窗口。 记录不同股票代码每 5 分钟的推文量。

标签沿用官方 combined_windows.json，与测试段相交共 22 个窗口，原边界不变。其中 16 条测试序列有异常窗口，另 5 条没有窗口，仍纳入误报评价。来源清单列出全部候选、原始 URL、哈希和排除原因。

1.2 问题动机与设计目标

不同业务序列的周期性与异常持续时间存在差异。部分序列具有日、周规律，另一些周期相关性较弱，单一相位模板难以同时适配。逐点残差侧重响应瞬时偏离，但容易产生较多离散告警；长跨度指数加权移动平均（EWMA）能够平滑短期波动、累积持续偏移，也可能弱化短脉冲。提高阈值虽能压低误报，却可能降低召回。

本项目据此设计训练期周期/稳健自回归分流、创新残差与局部水平联合评分，以及快慢双预设互补融合。系统将单路检测核心与数据库适配解耦，在 IoTDB 中提供 Java UDTF，并在统一的合成异常校准协议下，对比四种文献基线和内部消融变体。

2. 算法介绍与系统实现

2.1 周期点基线与自适应分支 A

周期 P 在训练段可支持的日、周候选中，按滞后相关性择优确定。基础周期点检测在时刻 t 取同相位最近 K=3 个历史缓存值的中位数作为基线 b_t，残差为 $r_t=x_t-b_t$。中心 m_t 和尺度 s_t 来自当前时刻之前最近 P 个全局真实残差的中位数与 MAD，而非按每个相位独立估计；尺度基于 1.4826 × MAD [6]，并设正下界。点分数为：

$$score_t = |r_t - m_t| / s_t$$

完成评分后，同相位缓存按 2 倍残差尺度进行有界更新，原始残差照常写入全局残差窗口，不因报警或标签剔除样本。

自适应分支 A 按训练期相关性 0.5 分流，弱周期序列仍全部保留：

- 强周期分支：3 条序列。 使用同期 K=3 中位数预测及过去 P 个残差的中位数/MAD 标准化。
- 稳健 AR 分支：18 条序列。 Python 用训练段拟合固定系数，Java 在线执行因果预测。滞后集合为 unique(1,2,3,6,12,P,2P)，各维共享训练序列的中位数与稳健尺度，标准化后限幅到 ±6。最多抽取 5,000 个等距训练目标点，采用 8 轮 Huber 迭代重加权最小二乘 [8]，阈值为 1.345，岭正则系数为 0.001×n，截距不惩罚。

受预测残差与水平变化分析视角启发 [9]，本项目为 AR 分支设计创新偏离 zAR 与局部水平偏离 zLevel 的联合评分。前者依据过去 P 个预测残差，后者依据过去 P 个原始值；两者各自分母的下界为相应训练发射尺度中位数的一半。联合偏离量为：

$$u_t = \mathrm{sign}(zAR_t) \times \sqrt{(|zAR_t| \times |zLevel_t|)}$$

该公式是项目设计，文献 [9] 为动机参考，未复现其完整干预检验。强周期分支的 u_t 则使用带符号的标准化残差。

强周期分支在输入限幅的 EWMA [7] 中选择：跨度 6、限幅 6，或跨度 12、限幅 4；AR 分支额外包含逐点候选。递推为 $E_t=(1-\alpha)E_{t-1}+\alpha u_t$，其中 $\alpha=2/(\mathrm{span}+1)$，输入先按候选设置限幅。各候选再以校准背景绝对分数的 99% 分位数归一化，选定模式与阈值 τA。AR 遇到时间戳缺口或非有限输入时，清空相关历史并重新热身。

2.2 周期分支 B 与互补融合

周期辅助分支 B 有两种预设：快速预设 Pscore 直接使用周期点分数；慢速预设 Sscore 使用跨度 12、限幅 4 的周期 EWMA，不含分支 A 的学习尺度下界与 q99 归一化。各预设阈值 τP、τS 同样由校准段确定。

参考异常检测集成的一般思路 [10]，本项目按各路阈值对齐分数后取算术均值，形成两个候选：

$$\mathrm{Fast} = (A/\tau A + \mathrm{Pscore}/\tau P) \,/\, 2$$

$$\mathrm{Slow} = (A/\tau A + \mathrm{Sscore}/\tau S) \,/\, 2$$

比值不是概率，均值融合也不是要求两路同时越限的硬 AND。最终报警阈值不低于 1，仅按校准背景报警预算提高。每条序列依据三套注入的平均事件 F2，在快、慢模式间选定一个；最终为 18 条慢模式、3 条快模式。

2.3 独立基线与跨语言实现

四种外部基线均说明论文依据与实际适配：

1. Isolation Forest [1]、LOF [2]：使用 scikit-learn 1.7.2。IF 为 200 棵树、每树最多 256 样本；LOF 为 novelty 模式、20 邻居。尾窗候选为 1、8、24，仅训练段标准化和拟合，最多 5,000 个训练窗口。

2. Spectral Residual [3]：按论文公式独立实现因果尾窗版本，候选长度 64、128、256。末尾预测延拓不读取真实未来；这是 SR，不是 SR-CNN。

3. Left Matrix Profile [4–5]：在最近最多 2,048 个合法非重叠左侧起点中，精确搜索当前子序列的标准化最近邻距离，候选长度 16、48。未实现完整 DAMP 剪枝，也不引用其吞吐量。

实现环境为 Java 17、Maven 3.9.x、IoTDB 2.0.11、Python 3.12。detector-core 是纯 Java 核心；detector-udf 以 provided 方式依赖 Tree UDF API，每个实例维护独立状态，每次调用按参数输出 score/flag/event/active 中的一列；runner 支持 CSV 与数据库运行。Python 完成拟合、选参、双路融合和评价。当前融合仍在库外执行，未合入单一 UDF。

3. 实验协议与结果分析

3.1 合成异常校准与评价口径

所有方法使用相同的 40%/20%/40% 时序划分。模型训练不用真实异常标签；校准段构造短脉冲、持续偏移、常数平线三套合成异常，每套 6 处，幅度由训练尺度确定，短脉冲长度包含 1、2、4 点。

各方法独立选参，在背景点报警率 0.5%、背景事件约每 14 天 1 次（至少允许 1 次）、注入窗外点报警率 1% 的预算下，优先提高三套注入的平均事件 F2，并保留超额与回退记录。先保存全部序列的参数和模式，再进行真实窗口评价。逐序列拟合与选参只用训练和校准段；但 NAB 已在方法开发中反复使用，因此整个方案不是新盲测。

评价在共同可评分历史上，将间隔不超过 1 小时的报警点合并为事件，再按事件起点归属测试段与闭区间官方窗口。每窗首次命中为 TP，窗内重复及窗外事件为 FP，无命中窗口为 FN。事件 F1 为 2TP/(2TP+FP+FN)。该指标不是官方 NAB 加权分数，不执行 point adjustment。由于官方窗口较宽，窗外点 FPR 和 AP 作为窗口代理指标报告。

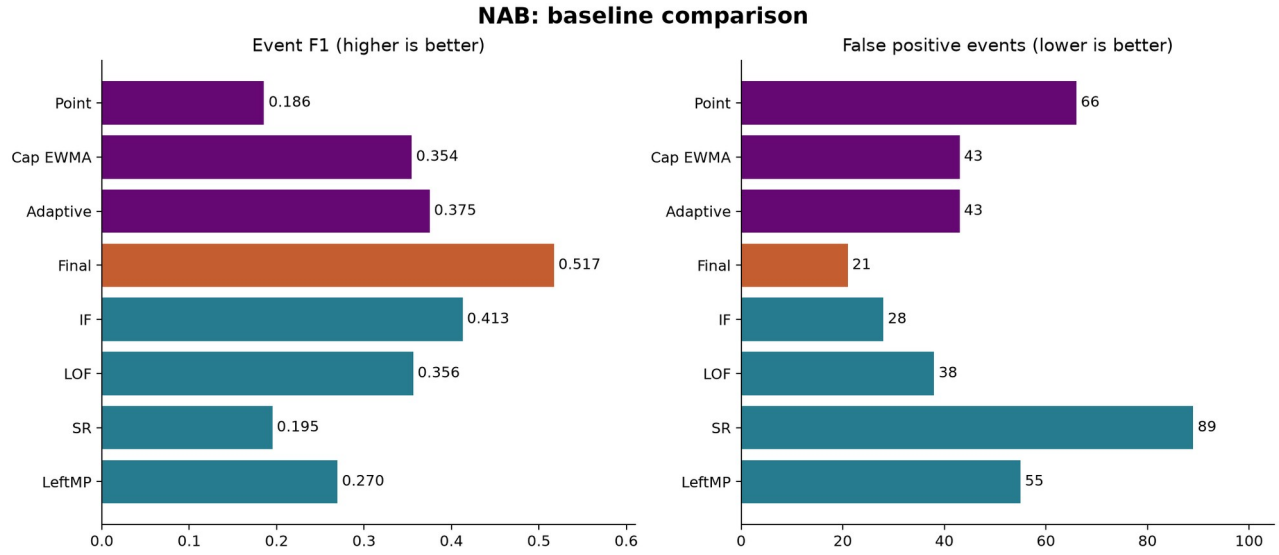
3.2 基线比较与消融结果

表 1 汇总最终融合、内部变体及四种文献基线在全部 21 条序列、22 个窗口上的结果。

方法	TP	FP	FN	精确率	召回率	事件 F1	窗外点 FPR
周期点检测	9	66	13	0.120	0.409	0.1856	0.34%
慢速周期 EWMA	14	43	8	0.246	0.636	0.3544	1.18%
自适应分支	15	43	7	0.259	0.682	0.3750	0.07%
固定慢速融合	15	23	7	0.395	0.682	0.5000	0.26%
固定快速融合	11	31	11	0.262	0.500	0.3438	0.15%

方法	TP	FP	FN	精确率	召回率	事件 F1	窗外点 FPR
最终快慢融合	15	21	7	0.417	0.682	0.5172	0.12%
Isolation Forest	13	28	9	0.317	0.591	0.4127	2.16%
LOF	13	38	9	0.255	0.591	0.3562	1.79%
SR	12	89	10	0.119	0.545	0.1951	0.11%
Left-MP	12	55	10	0.179	0.545	0.2697	0.15%

最终方法的 TP/FP/FN 为 15/21/7，精确率为 0.4167、召回率为 0.6818、事件 F1 为 0.5172。其中误报包括窗外 9 次、重复 12 次。全体事件 F1 高于本轮四个外部基线；相对最强外部对照 IF（F1=0.4127），多命中 2 个窗口、少产生 7 次误报。



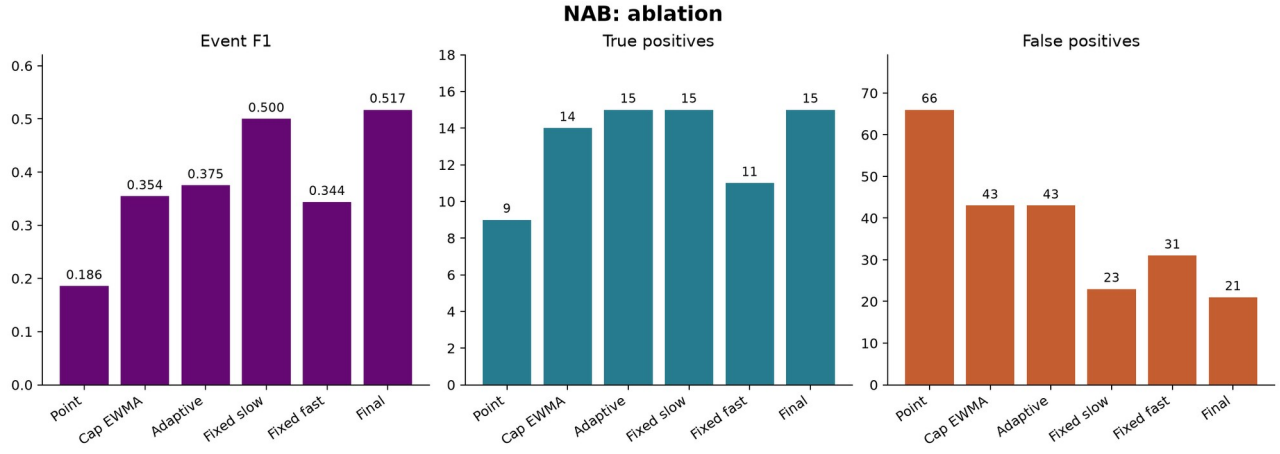
NAB 基线比较

内部变体体现了以下取舍：

1. 单路比较。 周期点检测命中 9 窗、误报 66 次；慢速周期分支命中 14 窗、误报 43 次；自适应分支 A 命中 15 窗、误报 43 次，对应 F1 分别为 0.1856、0.3544、0.3750。

2. 融合比较。 固定快速融合为 11 命中、31 误报，F1=0.3438；固定慢速融合为 15 命中、23 误报，F1=0.5000。验证选模式后再减少 2 次误报。相对 A，最终事件误报从 43 降至 21，减少 51.2%；总命中数仍为 15，但命中集合并非完全相同。

3. 分项指标。 最终窗外点 FPR 为 0.1206%，低于慢速周期的 1.1847% 和 IF 的 2.1604%，高于 A 的 0.0714%。AWS、Twitter 子组的融合 F1 分别为 0.5455、0.6667；仅两条序列的已知原因子组为 0.3810，低于 Left-MP 的 0.4348。未进行统计显著性检验。



NAB 融合消融

附录保留 22 窗口×10 种配置的 220 条记录，包括原始边界、首警、分数和阈值。两个例子说明具体判定差异：NAB-02（EC2 CPU 24ae8d 的第二个测试窗口）中，融合最高分/阈值为 1.027，并在窗内形成新事件；IF 为 0.926，未命中。NAB-17（Twitter FB）中，融合为 0.987，未越限；IF 为 1.014，命中。这里只解释分数与冻结阈值，不推断事故根因。若事件始于窗口之前，即便窗内仍有正点，也不计为该窗口的新 TP。

3.3 库内外一致性与组件成本

真实 IoTDB 对照中，AR 示例的 3,168 个有效评分点和周期示例的 8,976 点，在四种输出模式下均与库外 Java 逐行一致，最大误差为 0；两次独立查询也一致。已有 Java 检查、文献数学与因果性核验、默认兼容记录，以及 21 条序列无损分数回放的 231 项指标核对，详见独立《测试用例与实验描述》。

表 2 使用同一冻结 AR 模型，将公开 NAB 序列平铺为 1 万、10 万、100 万点，预热 2 轮、交替测量 5 轮，取中位数。平铺只构成成本负载，不增加精度实验的样本数。

输入点数	库内 UDF/ms	原值查询＋同 Java/ms
10,000	153.8	124.0
100,000	1,240.4	1,205.0
1,000,000	13,011.1	11,895.0

三规模的库内 UDF 耗时分别为 153.8、1240.4、13011.1 ms，原值查询后运行同一 Java 核心的耗时为 124.0、1205.0、11895.0 ms，输出校验和全部一致。两条路径都包含数据库取数；本实验未拆解查询、网络和序列化开销，也未测量 Python 融合总时延。实测 UDF 较慢，未观察到库内加速。

4. 反思与总结

本项目用训练期分流处理不同周期强度，用创新残差与局部水平联合评分构造自适应证据，再通过阈值归一化均值和快慢模式选择减少事件误报。在本轮 NAB 全体上，最终融合将自适应单路的 43 次误报降至 21 次，总命中数保持 15，事件 F1 高于四个文献基线。

结果仍有明确边界：22 窗中存在 7 个漏检；点级 FPR 未优于自适应单路，已知原因子组未优于 Left-MP；结论限于已参与开发的 NAB 三类场景；完整融合尚未进入 UDF。这些结果说明，较高的总体事件 F1 不能替代对子组、点指标和实现边界的检查。

后续应冻结当前方法与选参流程，在未参与开发的新时间段上检验；改善校准注入的代表性并分析报警延迟；将双路融合移入 Java UDF 后，单独测量完整查询的运行成本。

参考来源说明

正文编号对应 [1] Isolation Forest、[2] LOF、[3] SR、[4] Matrix Profile I、[5] DAMP、[6] Rousseeuw 与 Croux 的 MAD 工作、[7] Roberts 的 EWMA 工作、[8] Huber 稳健回归、[9] Tsay 的时序异常分析、[10] Aggarwal 与 Sathe 的异常集成专著、[11] Lavin 与 Ahmad 的 NAB 工作。完整书目、DOI、代码版本和实际适配见附录《论文与实现来源》。